Reporte

EQUIPO 13	
Nombre: Edgar Jesus Del Angel Martinez Israel Alejandro Pantoja Esquivel Andrea Fernanda Rodríguez Niño	**Matrícula:** 2160745 2155971 2160663
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo NetAcad: Unidad 3	**Actividad:** Diseño arquitectural del sistema
Fecha: 16/10/2026	

Bibliografía (formato APA)

- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Vernon, V. (2013). Implementing domain-driven design. Addison-Wesley.
- Documentación de PetMarket (Unidad 2): BRD, historias y criterios, BPMN y glosario.

Objetivo:

Registrar decisiones que condicionan las fronteras, relaciones, transacciones y responsabilidades de la arquitectura PetMarket, con fundamento en los criterios de aceptación del proceso de compra y cumplimiento de pedidos.

Procedimiento:

ARCHITECTURE DECISION RECORDS (ADR)

Los registros se mantienen como propuestas hasta que el equipo los apruebe. Cada decisión presenta alternativas viables, costos, consecuencias técnicas y evidencia del caso; los casos de uso y estados citados se conservan de Unidad 2.

ADR-000: Selección de Domain Model como patrón principal de lógica de negocio

Campo del ADR	ADR-000
Título	Selección de Domain Model como patrón principal de lógica de negocio
Estado	Propuesta técnica para aceptación formal del equipo
Contexto	PetMarket es una tienda digital de productos para mascotas que recibe pedidos con artículos distribuidos entre A1 y A2. Su proceso principal relaciona domicilio, entrega, pago, preparación por almacén, despacho, cancelación e incidencias. CA-05.03 exige que solo un pago aprobado habilite la preparación; CA-08.02 exige subórdenes por almacén; CA-15.01/03 condicionan cancelaciones por estado y autorización. La estructura elegida debe impedir que diferentes pantallas ejecuten transiciones inconsistentes.
Alternativas	Transaction Script: facilita operaciones lineales, pero obliga a repetir validaciones de pago, empaque y cancelación en procedimientos. Table Module: agrupa operaciones por tablas, útil para consultas y catálogos, aunque dificulta encapsular el ciclo de vida entre entidades y subórdenes. Domain Model: distribuye comportamientos en Pedido, OrdenPreparacion, Pago y Envio, con mayor esfuerzo inicial de diseño y pruebas, pero menos duplicación de reglas.
Decisión	Utilizar Domain Model como patrón principal. Los agregados conservan sus propias invariantes; los servicios de aplicación coordinan los casos de uso y los repositorios abstraen la persistencia. No se derivan clases automáticamente de las tablas del BRD; se diseñan cuando existe identidad, comportamiento o consistencia relevantes.
Consecuencias	Beneficios: reglas de estado testables, responsabilidades estables y mayor coherencia entre operaciones. Costos: mayor número de contratos, colaboración explícita y necesidad de controlar transacciones entre agregados. Riesgos: modelos anémicos o servicios excesivamente complejos. Control: pruebas de transiciones, revisión SOLID y trazabilidad por criterios de aceptación.
Trazabilidad	CA-05.02/03; CA-06.03; CA-08.02/03; CA-10.01; CA-12.01; CA-15.01/03. Historias: US-05, 06, 08, 10, 12, 15.

ADR-001: Límites y raíces de agregados con consistencia local

Campo del ADR	ADR-001
Título	Límites y raíces de agregados con consistencia local
Estado	Propuesta técnica para ratificación
Contexto	La relación entre venta, pago, inventario, preparación, envío y devolución no implica un único objeto transaccional. Un pedido contiene DetallePedido y una solicitud de cancelación; las subórdenes pertenecen al proceso particular de cada almacén. Un límite excesivamente amplio obligaría a persistir numerosos objetos ante cualquier cambio, dificultando concurrencia y seguimiento.
Alternativas	A: Un agregado gigante que incluya Pedido, Pago, Inventario, subórdenes y Envio; simplifica la navegación, pero acopla estados y bloquea evolución. B: Cada entidad como raíz independiente; ahorra composición, pero deja reglas sin un punto claro de autoridad. C: Agregados cohesionados: Pedido, Pago, Almacen, OrdenPreparacion, Envio y SolicitudDevolucion, con entidades internas y referencias entre raíces.
Decisión	Adoptar fronteras cohesionadas, con operaciones de cambio mediadas por su Aggregate Root. Pedido conserva total, estado y la solicitud de cancelación. OrdenPreparacion controla sus líneas y cambios de fase; Almacen controla existencias por SKU. Las relaciones entre raíces se expresan por IDs y las actualizaciones que atraviesan fronteras se coordinan.
Consecuencias	Beneficios: reduce acoplamiento, clarifica reglas y permite pruebas específicas. Costos: algunas operaciones requieren coordinación y lecturas adicionales. Riesgos: estados temporalmente distintos entre pedido y subórdenes; deben interpretarse por contrato, no suponerse sincronía automática. Control: eventos de dominio, consultas de estado y pruebas de casos de uso.
Trazabilidad	US-08/09/10/12/15. CA-08.02/03; CA-09.01/02; CA-10.01/02; CA-12.01; CA-15.01/03.

ADR-002: Referencias por identificador entre raíces de agregado

Campo del ADR	ADR-002
Título	Referencias por identificador entre raíces de agregado
Estado	Propuesta técnica para ratificación
Contexto	Una suborden requiere indicar su pedido y su almacén de origen. Un pago y un envío pertenecen al pedido que los origina, mientras que el catálogo reconoce productos mediante SKU. La navegación directa con objetos mutables entre agregados permitiría modificar estados desde fuera de las raíces autorizadas.
Alternativas	A: Asociaciones directas a objetos de otro agregado; simplifican algunos diagramas, pero admiten cambios laterales y dependencias fuertes. B: Copiar todos los datos en cada contexto; facilita lecturas, pero duplica estado de negocio y plantea reconciliación. C: Guardar pedidoId, almacenId y productoSKU como referencias externas, resolviendo información mediante puertos y DTO cuando hace falta.
Decisión	Las asociaciones entre agregados se mantienen como identificadores estables. Los campos internos de DetallePedido, LineaPreparacion e IncidenciaEntrega

	pertenecen a su raíz. Application Services resuelven colaboradores mediante interfaces en vez de navegar objetos ajenos o exponer referencias mutables.
Consecuencias	Beneficios: independencia del ciclo de vida y fronteras claras. Costos: más consultas o adaptadores para recuperar información. Riesgos: IDs inexistentes o desactualizados; la aplicación valida la referencia de manera explícita. Control: pruebas de recuperación, errores de recurso ausente y DTO de lectura.
Trazabilidad	CA-07.01/02; CA-08.02/03; CA-12.03; CA-16.02; US-07, 08, 12, 16.

ADR-003: Pago externo separado de transacciones locales

Campo del ADR	ADR-003
Título	Pago externo separado de transacciones locales
Estado	Propuesta técnica para ratificación
Contexto	El gateway externo comunica aprobación, rechazo o puede dejar una respuesta incierta. Las bases de datos propias no pueden incluir un cargo bancario en una transacción ACID local. Además, no corresponde generar subórdenes ante un pago rechazado, y el pedido debe conservar PENDIENTE_PAGO (CA-06.03).
Alternativas	A: Mantener una transacción local abierta durante el cobro; aumenta los tiempos de bloqueo y no ofrece rollback bancario. B: Tratar todo como una operación única sin registros intermedios; impide reconciliar un resultado tardío. C: Registrar pedido pendiente, ejecutar cobro fuera de la unidad de trabajo y persistir su respuesta en una transacción local posterior. Manejar timeout como resultado no confirmado.
Decisión	Seleccionar separación explícita entre fases locales y llamada al proveedor. Registrar identificación del intento, mantener pedido en PENDIENTE_PAGO si el cobro se rechaza o no se confirma y crear subórdenes solamente cuando exista aprobación. La integración real requiere idempotencia, conciliación y tratamiento de conflictos de stock posterior al cobro.
Consecuencias	Beneficios: estados observables y errores no confundidos con rechazos. Costos: diseño de recuperación y auditoría de intentos. Riesgos: cargo aprobado pero persistencia fallida, inventario alterado después de cobrar o duplicación por reintento. El prototipo simula la separación, no promete reembolsos automáticos ni pagos productivos. Control: pruebas de fases, rechazo, timeout y consistencia.
Trazabilidad	CA-05.02/03; CA-06.01/02/03; US-05/06; casos CU-01/CU-02.

ADR-004: Separación de orquestación, reglas y adaptadores

Campo del ADR	ADR-004
Título	Separación de orquestación, reglas y adaptadores
Estado	Propuesta técnica para ratificación
Contexto	Los casos de uso incluyen consulta de stock, compra, distribución entre almacenes, autorización de cancelaciones, preparación y seguimiento. Si cada servicio de aplicación calcula directamente los estados, cantidades y condiciones de negocio, esas reglas se duplicarían en APIs y tareas administrativas.

Alternativas	**A: Controladores y scripts que contienen persistencia, cálculos y decisiones; desarrollo inicial rápido, pero baja capacidad de prueba. B: Servicios de aplicación con lógica empresarial propia; reduce controladores, aunque transforma la Service Layer en un segundo dominio. C: Servicios con puertos, agregados y adaptadores: reciben entradas, recuperan objetos, invocan operaciones de dominio y devuelven DTO o errores interpretables.**
Decisión	**Mantener Application Services orientados a casos de uso. El dominio protege reglas como estados cancelables y empaque desde PREPARADO; los repositorios aportan persistencia por interfaces; los adaptadores en memoria y el gateway simulado se sustituyen sin modificar el dominio. Commands modifican estado y Queries devuelven proyecciones de lectura.**
Consecuencias	**Beneficios: mejor separación de responsabilidades, inversión de dependencias y pruebas deterministas. Costos: contratos y DTO adicionales. Riesgos: proliferación injustificada de servicios o inconsistencias de nombre entre UML y código; se controlan con convenciones, pruebas y revisión de contratos. La arquitectura no implica una aplicación web desplegada.**
Trazabilidad	**CA-02.01/03; CA-05.02; CA-08.02/03; CA-09.01; CA-15.03; CA-17.01; etapas de Repository/UoW, CQRS y SOLID.**